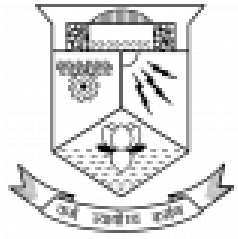
LABORATORY RECORD

Submitted to the
APJ Abdul Kalam Technological University
in partial fulfilment of the requirements
for the award of the Degree of
MASTER OF TECHNOLOGY
in
ELECTRONICS and COMMUNICATION
with specialisation in
Signal Processing

Submitted by
GOKUL PRAKASH(TVE24ECSP07)
Subject Code: 221LEC001
Subject Name: SIGNAL PROCESSING LAB 1
2025-2026



Department of Electronics and
Communication
College of Engineering Trivandrum
Thiruvananthapuram

Course PO mapping

	221LEC001 Signal Processing Lab-1		PO1	PO2	PO3	PO4	PO5	PO6
CO1	Apply the principles of linear algebra and random processes in signal processing applications, analyze observations of experiments/simulations and infer.		0	0	3	0	0	3
CO2	Implement algorithms in machine learning.		0	0	3	0	0	3
CO3	Design and implement a real world applications of signal processing.		0	3	0	0	0	3
		Course PO mapping	0	3	3	0	0	3

Syllabus

No	Topics
1	**Linear Algebra**
1.1	Row Reduced Echelon Form: To reduce the given mxn matrix into Row reduced Echelon form
1.2	Gram-Schmidt Orthogonalization: To find orthogonal basis vectors for the given set of vectors. Also find orthonormal basis.
1.3	Least Squares Fit to a Sinusoidal function
1.4	Least Squares fit to a quadratic polynomial
1.5	Eigen Value Decomposition
1.6	Singular Value Decomposition
1.7	Karhunen- Loeve Transform
2	**Advanced DSP**
2.1	Sampling rate conversion: To implement Down sampler and Up sampler and study their characteristics
2.2	Two channel Quadrature Mirror Filterbank: Design and implement a two channel Quadrature Mirror Filterbank
3	**Random Processes**
3.1	To generate random variables having the following probability distributions (a) Bernoulli(b) Binomial(c) Geometric(d) Poisson(e)Uniform,(f) Gaussian(g)Exponential (h) Laplacian
3.2	Central Limit Theorem: To verify the sum of sufficiently large number of Uniformly distributed random variables is approximately Gaussian distributed and to estimate the probability density function of the random variable.

4	**Machine Learning**
4.1	Implementation of K Nearest Neighbours Algorithm with decision region plots
4.2	Implementation of K Means Algorithm with decision region plots
4.3	Implementation of Perceptron Learning Algorithms with decision region plots
4.4	Implementation of SVM algorithm for classification applications
5	**Implement a mini project pertaining to an application of Signal Processing in real life, make a presentation and submit a report**

Experiment 1

Reduction of a Matrix to Row Reduced Echelon Form

Code:

```
import numpy as np
R=int(input("Enter the number of rows: "))
C=int(input("Enter the number of columns: "))
matrix=[[0 for c in range(C)] for r in range(R)]
for r in range(R):
    for c in range(C):
        matrix[r][c]=int(input(f"Enter element at row {r + 1}, column {c + 1}:"))
matrix_array=np.array(matrix)
print("The given matrix is:\n",matrix_array)
for r in range(R):
    pivot=matrix_array[r][r]
    if pivot==0:
        for i in range(r+1,R):
            if matrix_array[i][r]!=0:
                matrix_array[[r,i]]=matrix_array[[i,r]]
                pivot=matrix_array[r][r]
            break
    if pivot!=0:
        matrix_array[r]=matrix_array[r]/pivot
        for i in range(R):
            if i!=r:
                factor=matrix_array[i][r]
                matrix_array[i]-=factor*matrix_array[r]
print("\nReduced Row Echelon Form (RREF):\n",matrix_array)
```

```
count=0
for i in range(R):
    if np.all(matrix_array[i,:]==0):
        count+=1
rank=R-count
print("Rank of matrix=",rank)
```

```
Enter the number of rows: 3
Enter the number of columns: 4
Enter element at row 1, column 1:1
Enter element at row 1, column 2:2
Enter element at row 1, column 3:-1
Enter element at row 1, column 4:3
Enter element at row 2, column 1:2
Enter element at row 2, column 2:4
Enter element at row 2, column 3:-2
Enter element at row 2, column 4:6
Enter element at row 3, column 1:3
Enter element at row 3, column 2:6
Enter element at row 3, column 3:-3
Enter element at row 3, column 4:9
The given matrix is:
 [[ 1  2 -1  3]
 [ 2  4 -2  6]
 [ 3  6 -3  9]]

Reduced Row Echelon Form (RREF):
 [[ 1  2 -1  3]
 [ 0  0  0  0]
 [ 0  0  0  0]]
Rank of matrix= 1
```

Experiment 2

Gram-Schmidt Orthogonalization

Code:

```
# Input: Number of vectors
k = int(input("Enter the number of vectors (k): "))
# Step 1: Gather the input vectors
input_vectors = []
for i in range(k):
    vector = [float(x) for x in input(f"Enter vector {i + 1} as space-separated values: ").split()]
    input_vectors.append(vector)
    print(f"Vector {i + 1}: {vector}")
# Step 2: Perform Gram-Schmidt Orthogonalization
orthogonal_basis = []
orthonormal_sets = []
for vector in input_vectors:
    for basis_vector in orthogonal_basis:
        # Calculate projection of vector onto basis_vector
        dot_product = sum(v1 * v2 for v1, v2 in zip(vector, basis_vector))
        projection = [dot_product / sum(vi ** 2 for vi in basis_vector) * bi for bi in basis_vector]
        # Subtract projection from vector
        vector = [v - p for v, p in zip(vector, projection)]
    orthogonal_basis.append(vector)
# Step 3: Normalize the orthogonal basis to get orthonormal sets
for basis_vector in orthogonal_basis:
    magnitude = sum(x ** 2 for x in basis_vector) ** 0.5
    if magnitude != 0:  # Prevent division by zero for zero vectors
        orthonormal_sets.append([x / magnitude for x in basis_vector])
# Output Results
print(f"\nThe orthogonal basis vectors are: {orthogonal_basis}")
print(f"The orthonormal sets are: {orthonormal_sets}")
```

```
Enter the number of vectors (k): 3
Enter vector 1 as space-separated values: 3 0 4
Vector 1: [3.0, 0.0, 4.0]
Enter vector 2 as space-separated values: -1 0 7
Vector 2: [-1.0, 0.0, 7.0]
Enter vector 3 as space-separated values: 2 9 11
Vector 3: [2.0, 9.0, 11.0]

The orthogonal basis vectors are: [[3.0, 0.0, 4.0], [-4.0, 0.0, 3.0], [0.0, 9.0, 0.0]]
The orthonormal sets are: [[0.6, 0.0, 0.8], [-0.8, 0.0, 0.6], [0.0, 1.0, 0.0]]
```

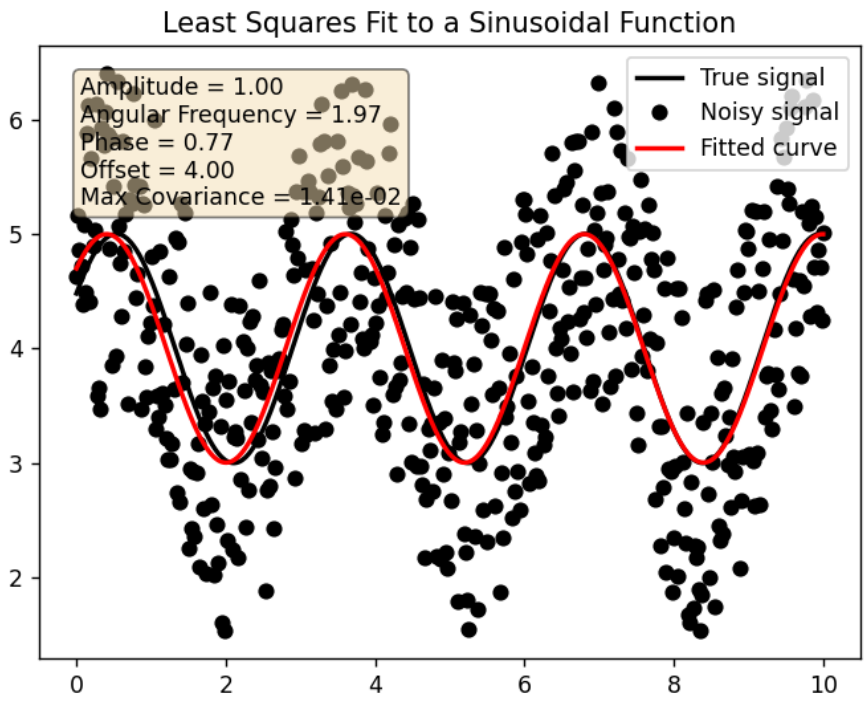
Experiment 3

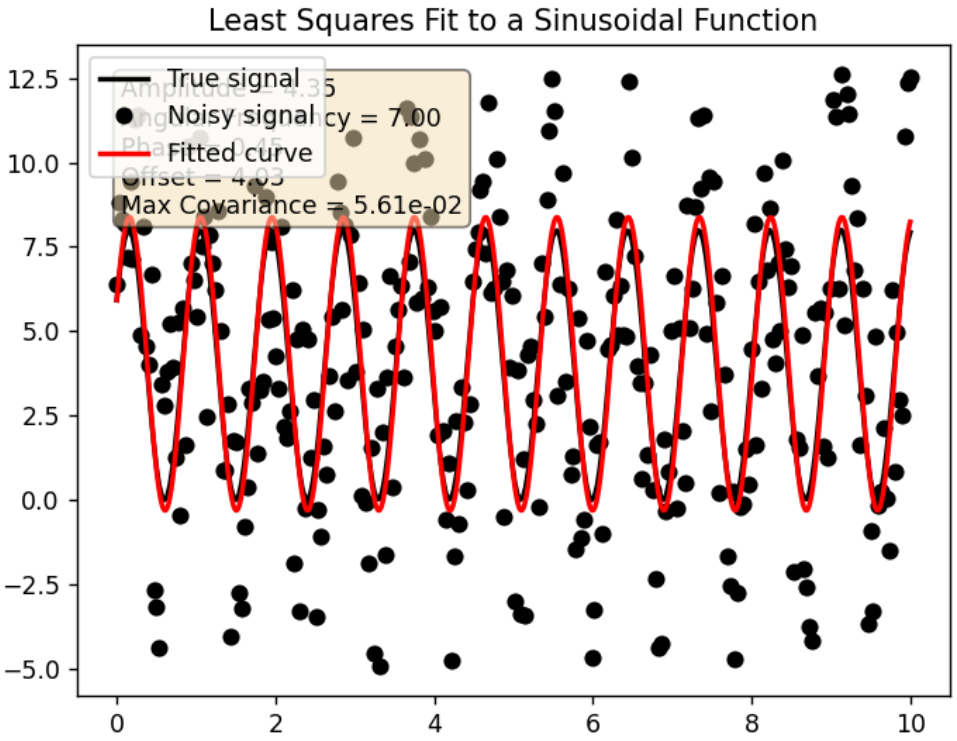
Least Squares Fit to a Sinusoidal Function

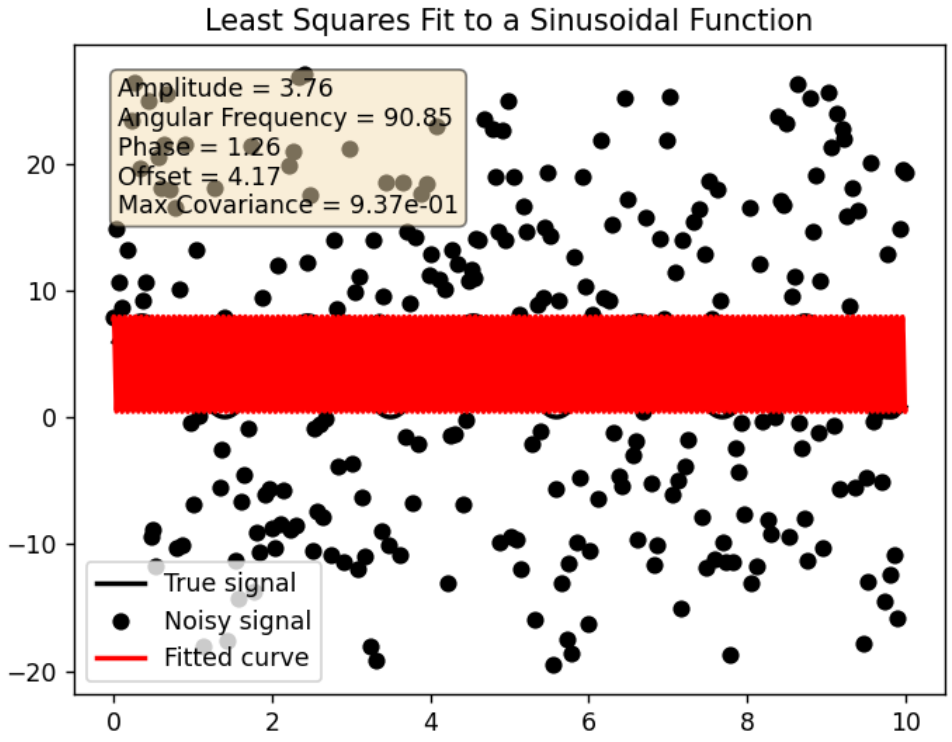
Code:

```
import numpy as np
import scipy.optimize
import matplotlib.pyplot as plt
def fit_sin(tt,yy):
    tt=np.array(tt)
    yy=np.array(yy)
    ff=np.fft.fftfreq(len(tt),(tt[1]-tt[0]))
    Fyy=abs(np.fft.fft(yy))
    guess_freq=abs(ff[np.argmax(Fyy[1:])+1])
    guess_amp=np.std(yy)*2.**0.5
    guess_offset=np.mean(yy)
    guess=np.array([guess_amp,2.*np.pi*guess_freq,0.,guess_offset])
    def sinfunc(t,A,w,p,c):
        return A*np.sin(w*t+p)+c
    popt,pcov=scipy.optimize.curve_fit(sinfunc,tt,yy,p0=guess)
    A,w,p,c=popt
    f=w/(2.*np.pi)
    fitfunc=lambda t:A*np.sin(w*t+p)+c
    return
{"amp":A,"omega":w,"phase":p,"offset":c,"freq":f,"period":1./f,"fitfunc":fitfunc,"maxcov":np.max(pcov),
"rawres":(guess,popt,pcov)}
N,amp,omega,phase,offset,noise=500,1.,2.,.5,4.,3
tt=np.linspace(0,10,N)
tt2=np.linspace(0,10,10*N)
```

```
yy=amp*np.sin(omega*tt+phase)+offset

yynoise=yy+noise*(np.random.random(len(tt))-0.5)

res=fit_sin(tt,yynoise)

print(f"Amplitude={res['amp']}, Angular freq.={res['omega']}, Phase={res['phase']}, Offset={res['offset']}, Max. Cov.={res['maxcov']}")

plt.title("Least Squares Fit to a Sinusoidal Function")

plt.plot(tt,yy,"-k",label="True signal",linewidth=2)

plt.plot(tt,yynoise,"ok",label="Noisy signal")

plt.plot(tt2,res["fitfunc"](tt2),"r-",label="Fitted curve",linewidth=2)

plt.legend(loc="best")

info_text=(f"Amplitude = {res['amp']:.2f}\n"f"Angular Frequency = {res['omega']:.2f}\n"f"Phase = {res['phase']:.2f}\n"f"Offset = {res['offset']:.2f}\n"f"Max Covariance = {res['maxcov']:.2e}")

plt.text(0.05,0.95,info_text,transform=plt.gca().transAxes,fontsize=10,verticalalignment='top',bbox=dict(boxstyle='round',facecolor='wheat',alpha=0.5))

plt.show()
```




Least Squares Fit to a Sinusoidal Function
True signal
Noisy signal
Fitted curve
Offset = 4.03
Max Covariance = 5.61e-02
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
0
2
4
6
8
10


Least Squares Fit to a Sinusoidal Function
Amplitude = 3.76
Angular Frequency = 90.85
Phase = 1.26
Offset = 4.17
Max Covariance = 9.37e-01
True signal
Noisy signal
Fitted curve
20
10
0
−10
−20
0
2
4
6
8
10

Experiment 4

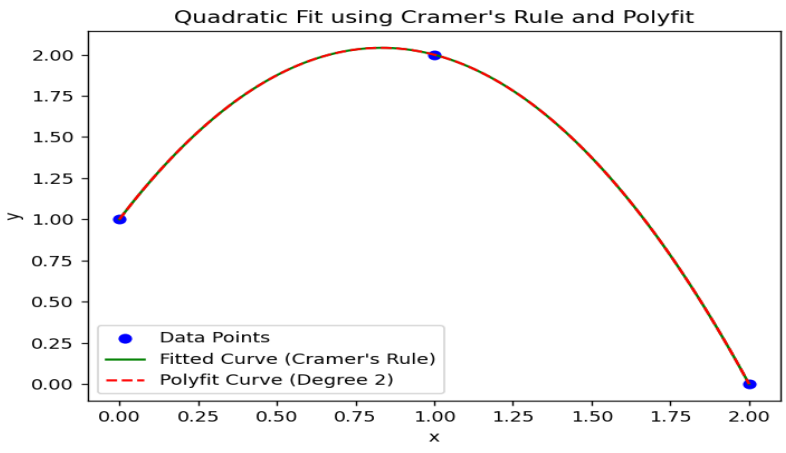
Least Squares Fit to a Quadratic Polynomial

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Define matrix A and vector b
A = np.array([[0, 1], [1, 1], [2, 1]])
b = np.array([1, 2, 0])

# Create feature matrix with constant, linear, and quadratic terms
ones = np.ones(A.shape[0])
x_feature = A[:, 0]
squared_feature = x_feature ** 2
features = np.column_stack((ones, x_feature, squared_feature))

# Calculate the determinant of the original feature matrix
m_det = np.linalg.det(features)
if np.isclose(m_det, 0):
    raise ValueError("The determinant of the feature matrix is zero, indicating a singular matrix.")

# Calculate determinants by replacing each column with vector b, using Cramer's Rule
determinants = []
for i in range(features.shape[1]):
    modified_features = features.copy()
    modified_features[:, i] = b  # Replace column i with vector b
    det = np.linalg.det(modified_features)
    determinants.append(det / m_det)
```

```
# Coefficients of the quadratic fit
coefficients = np.array(determinants)
print("Coefficients:", coefficients)

# Plot the data points
plt.scatter(x_feature, b, label="Data Points", color="blue")

# Plot the fitted curve using the calculated coefficients
u = np.linspace(0, 2, 100)
fitted_curve = coefficients[2] * u**2 + coefficients[1] * u + coefficients[0]
plt.plot(u, fitted_curve, label="Fitted Curve (Cramer's Rule)", color="green")

# Compare with numpy's polyfit
polyfit_coeffs = np.polyfit(x_feature, b, 2)
plt.plot(u, np.polyval(polyfit_coeffs, u), 'r--', label="Polyfit Curve (Degree 2)")
# Final plot adjustments
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.title("Quadratic Fit using Cramer's Rule and Polyfit")
plt.show()
```

OUTPUT



Experiment 5

Eigen Value Decomposition

Code:

```
import numpy as np
def eigen_decomposition(A, max_iterations, tolerance):
    n = A.shape[0]
    eigenvectors = np.eye(n)
    eigenvalues = np.zeros(n)
     for k in range(n):
        # Start with a random vector
        x = np.random.randn(n)
        x = x / np.linalg.norm(x)

        # Power iteration
        for iteration in range(max_iterations):
            y = A @ x
            lambda_ = x @ y
            x = y / np.linalg.norm(y)

            # Check convergence
            if np.linalg.norm(A @ x - lambda_ * x) < tolerance:
                break

        eigenvalues[k] = lambda_
        eigenvectors[:, k] = x
```

```
        # Deflate the matrix
        A = A - lambda_ * np.outer(x, x)

    return eigenvalues, eigenvectors

# Simplified matrix input
def input_matrix():
    print("Enter the matrix row by row, elements separated by spaces.")
    rows = int(input("How many rows in the matrix? "))
    matrix = []

    for i in range(rows):
        row = list(map(float, input(f"Enter row {i + 1}: ").split()))
        matrix.append(row)

    return np.array(matrix)

# Get input matrix
A = input_matrix()

max_iterations = 1000
tolerance = 1e-6

# Perform eigen decomposition
eigenvalues, eigenvectors = eigen_decomposition(A, max_iterations, tolerance)
```

Output results

print('\nEigenvectors:')

print(eigenvectors)

print('Eigenvalues:')

print(eigenvalues)

```
Enter the matrix row by row, elements separated by spaces.
How many rows in the matrix? 2
Enter row 1: 4 1
Enter row 2: 1 3

Eigenvectors:
[[ 0.85065098  0.52573059]
 [ 0.52573084 -0.85065113]]
Eigenvalues:
[4.61803399 2.38196601]

Process finished with exit code 0
```

Experiment 6

Singular Value Decomposition

Code:

```
import numpy as np
# Function to get matrix input from the user
def get_matrix():
    rows = int(input("Enter the number of rows: "))
    cols = int(input("Enter the number of columns: "))

    print("Enter the matrix elements row by row (separated by spaces):")
    matrix = []
    for i in range(rows):
        row = list(map(float, input(f"Row {i + 1}: ").split()))
        if len(row) != cols:
            print("Error: Number of elements does not match the number of columns.")
            return None
        matrix.append(row)
    return np.array(matrix)

# Get matrix input
A = get_matrix()
if A is None:
    print("Invalid matrix input.")
else:
    # Perform Singular Value Decomposition
    U, S, Vt = np.linalg.svd(A)
```

```
# Display the results
print("\nMatrix A:")
print(A)
print("\nLeft singular vectors (U):")
print(U)
print("\nSingular values (Sigma):")
print(S)
print("\nRight singular vectors (V^T):")
print(Vt)

# Reconstruct the original matrix using U, Sigma, and Vt
Sigma = np.zeros((A.shape[0], A.shape[1]))
np.fill_diagonal(Sigma, S)
A_reconstructed = U @ Sigma @ Vt

print("\nReconstructed Matrix A (using U, Sigma, V^T):")
print(A_reconstructed)
```

```
Enter the number of rows: 2
Enter the number of columns: 2
Enter the matrix elements row by row (separated by spaces):
Row 1: 1 2
Row 2: 3 4

Matrix A:
[[1. 2.]
 [3. 4.]]

Left singular vectors (U):
[[-0.40455358 -0.9145143 ]
 [-0.9145143   0.40455358]]

Singular values (Sigma):
[5.4649857  0.36596619]

Right singular vectors (V^T):
[[-0.57604844 -0.81741556]
 [ 0.81741556 -0.57604844]]

Reconstructed Matrix A (using U, Sigma, V^T):
[[1. 2.]
 [3. 4.]]
```

Experiment 7

Karhunen-Loève Transform (KLT)

Code:

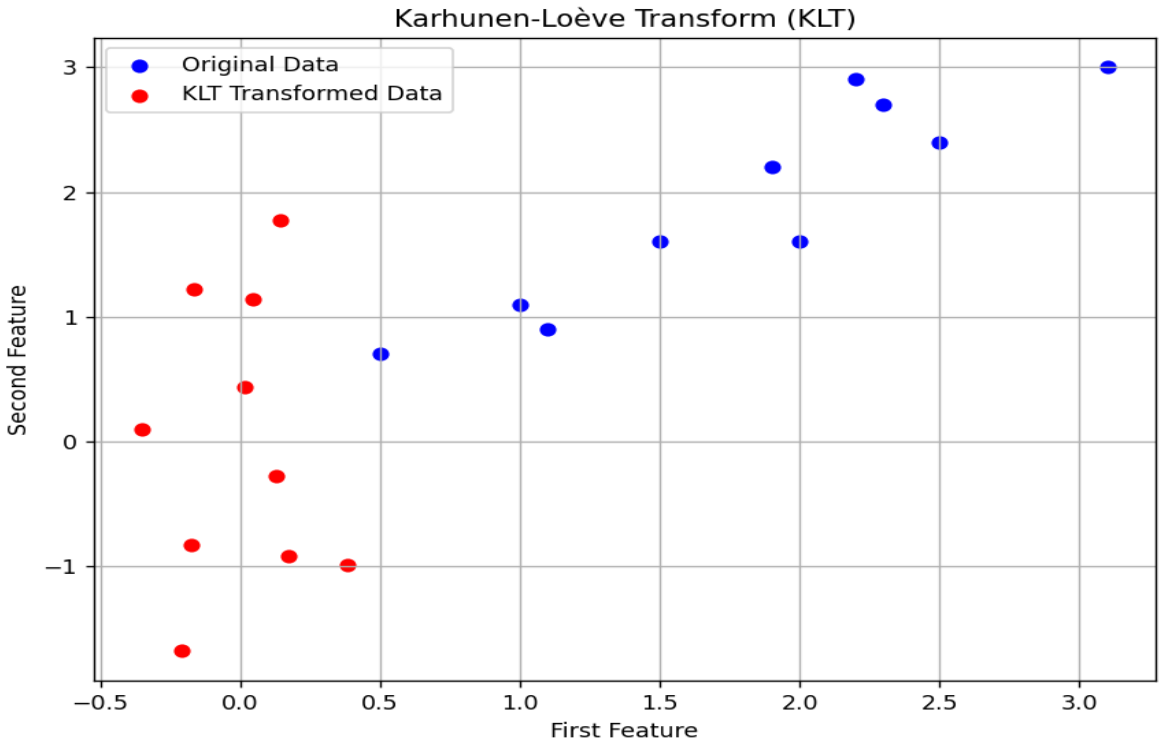
```
import numpy as np
import matplotlib.pyplot as plt

# Define a sample dataset (2D points)
X = np.array([
    [2.5, 2.4],
    [0.5, 0.7],
    [2.2, 2.9],
    [1.9, 2.2],
    [3.1, 3.0],
    [2.3, 2.7],
    [2.0, 1.6],
    [1.0, 1.1],
    [1.5, 1.6],
    [1.1, 0.9]
])

# Step 1: Center the data
X_mean = np.mean(X, axis=0)
X_centered = X - X_mean
```

```
# Step 2: Compute the covariance matrix
cov_matrix = np.cov(X_centered.T)

# Step 3: Perform eigen decomposition
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Step 4: Project the data onto eigenvectors
X_transformed = X_centered @ eigenvectors

# Display results
print("Original Data:\n", X)
print("\nMean of Data:\n", X_mean)
print("\nCovariance Matrix:\n", cov_matrix)
print("\nEigenvalues:\n", eigenvalues)
print("\nEigenvectors:\n", eigenvectors)
print("\nTransformed Data:\n", X_transformed)

# Plot original and transformed data
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], color='blue', label='Original Data')
plt.scatter(X_transformed[:, 0], X_transformed[:, 1], color='red', label='KLT Transformed Data')
plt.title("Karhunen-Loève Transform (KLT)")
plt.xlabel("First Feature")
plt.ylabel("Second Feature")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

```
Original Data:
 [[2.5 2.4]
 [0.5 0.7]
 [2.2 2.9]
 [1.9 2.2]
 [3.1 3. ]
 [2.3 2.7]
 [2.  1.6]
 [1.  1.1]
 [1.5 1.6]
 [1.1 0.9]]

Mean of Data:
 [1.81 1.91]

Covariance Matrix:
 [[0.61655556 0.61544444]
 [0.61544444 0.71655556]]

Eigenvalues:
 [0.0490834  1.28402771]

Eigenvectors:
 [[-0.73517866 -0.6778734 ]
 [ 0.6778734  -0.73517866]]
```

```
Eigenvectors:
 [[-0.73517866 -0.6778734 ]
 [ 0.6778734  -0.73517866]]

Transformed Data:
 [[-0.17511531 -0.82797019]
 [ 0.14285723  1.77758033]
 [ 0.38437499 -0.99219749]
 [ 0.13041721 -0.27421042]
 [-0.20949846 -1.67580142]
 [ 0.17528244 -0.9129491 ]
 [-0.3498247   0.09910944]
 [ 0.04641726  1.14457216]
 [ 0.01776463  0.43804614]
 [-0.16267529  1.22382056]]
```



Experiment 8

Sampling rate conversion

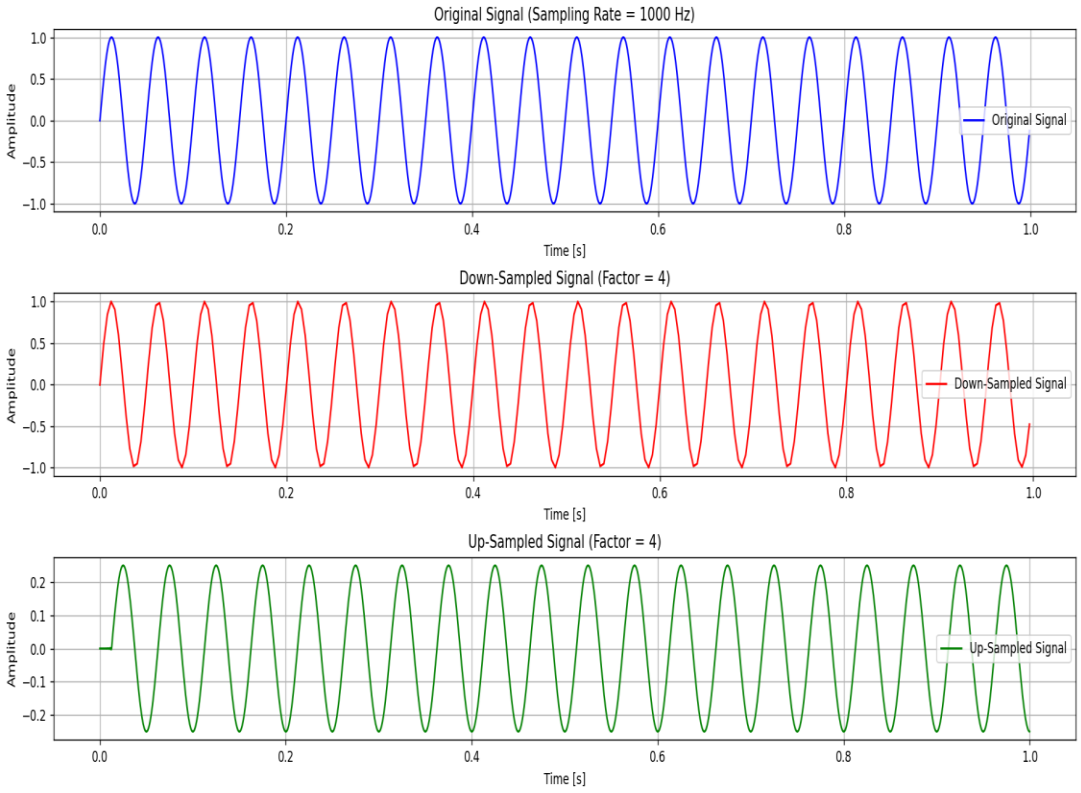
Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import decimate, firwin, lfilter

# Generate a sample signal (sine wave)
fs = 1000  # Original sampling rate (Hz)
t = np.arange(0, 1, 1/fs)  # 1 second of data
f = 50  # Frequency of the sine wave
x = np.sin(2 * np.pi * f * t)  # Sine wave signal

# --- Down-Sampling ---
down_factor = 4  # Down-sampling factor
downsampled_signal = decimate(x, down_factor)  # Decimate by factor

# Time for the downsampled signal
t_downsampled = t[::down_factor]

# --- Up-Sampling ---
up_factor = 4  # Up-sampling factor
# Zero stuffing (insert zeros between the samples)
upsampled_signal = np.zeros(len(x) * up_factor)
upsampled_signal[::up_factor] = x

# Low-pass filter to smooth the signal after upsampling
```

```
nyquist_rate = fs / 2
cutoff = nyquist_rate / up_factor  # The cutoff frequency should be half of the up-sampled rate
num_taps = 101  # Number of taps for the FIR filter
low_pass_filter = firwin(num_taps, cutoff / nyquist_rate)

# Apply filter to smooth the upsampled signal
upsampled_signal_filtered = lfilter(low_pass_filter, 1.0, upsampled_signal)

# Time for the upsampled signal
t_upsampled = np.arange(0, len(upsampled_signal_filtered)) / (fs * up_factor)

# Plot all three signals (Original, Down-Sampled, Up-Sampled)
plt.figure(figsize=(10, 8))

# Original Signal Plot
plt.subplot(3, 1, 1)
plt.plot(t, x, label='Original Signal', color='b')
plt.title("Original Signal (Sampling Rate = 1000 Hz)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()

# Down-Sampled Signal Plot
plt.subplot(3, 1, 2)
plt.plot(t_downsampled, downsampled_signal, label='Down-Sampled Signal', color='r')
plt.title(f"Down-Sampled Signal (Factor = {down_factor})")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()
```

```
# Up-Sampled Signal Plot
plt.subplot(3, 1, 3)
plt.plot(t_upsampled, upsampled_signal_filtered, label='Up-Sampled Signal', color='g')
plt.title(f"Up-Sampled Signal (Factor = {up_factor})")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT:



Experiment 9

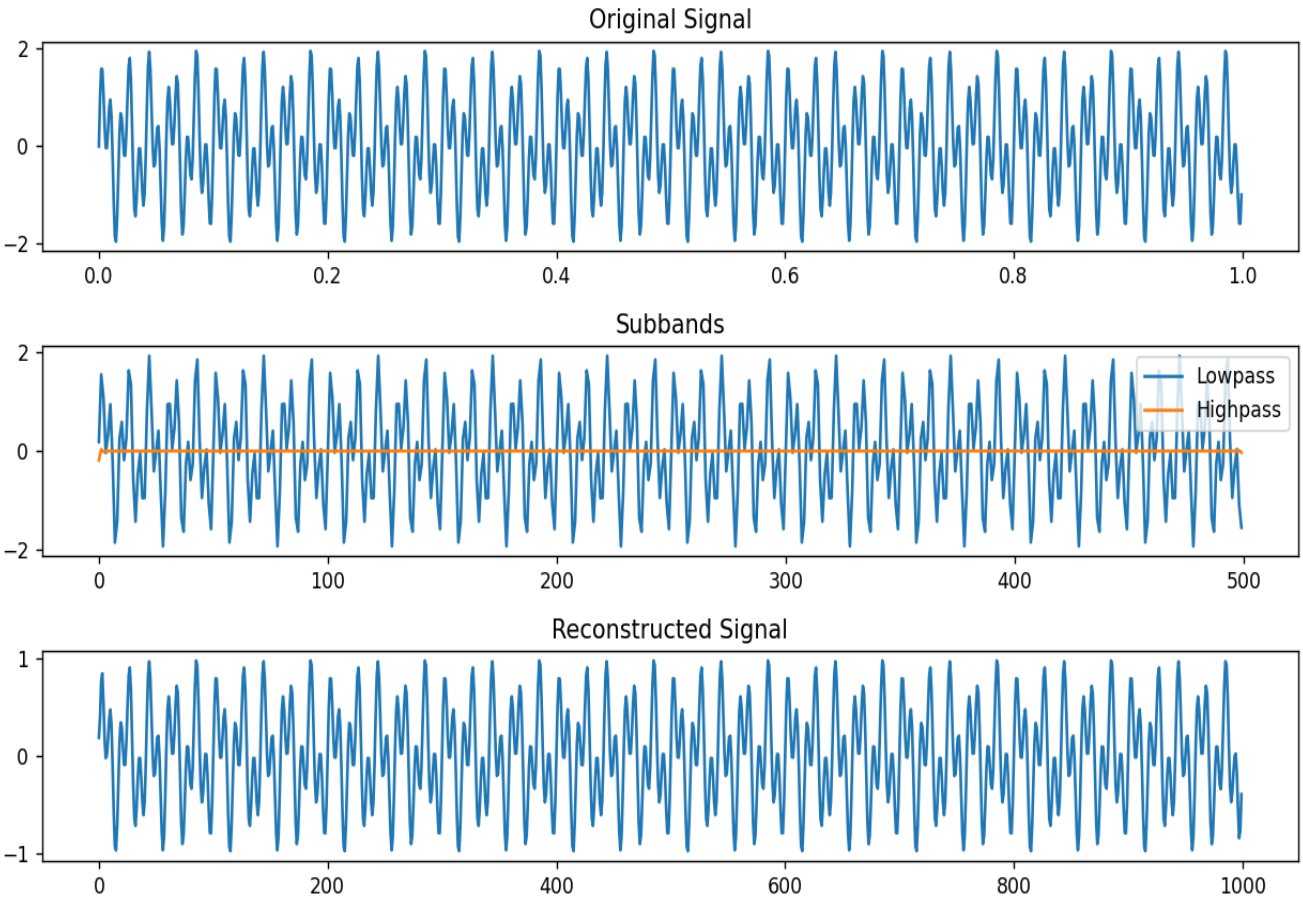
Two channel Quadrature Mirror Filterbank: Design and implement a two channel Quadrature Mirror Filterbank

Code:

```
import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt

def qmf_filter_design(N, beta=5.0):
    H0 = signal.firwin(N + 1, 0.5, window=('kaiser', beta))
    H1 = (-1) ** np.arange(N + 1) * H0
    return H0, H1

def analysis_filterbank(input_signal, H0, H1):
    low = signal.convolve(input_signal, H0, mode='same')[::2]
    high = signal.convolve(input_signal, H1, mode='same')[::2]
    return low, high

def synthesis_filterbank(low, high, H0, H1):
    up_low = np.zeros(len(low) * 2)
    up_low[::2] = low
    up_high = np.zeros(len(high) * 2)
    up_high[::2] = high
    recon_low = signal.convolve(up_low, H0, mode='same')
    recon_high = signal.convolve(up_high, -H1, mode='same')
    return recon_low + recon_high
```

```
fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
x = np.sin(2 * np.pi * 50 * t) + np.sin(2 * np.pi * 120 * t)

N = 64
H0, H1 = qmf_filter_design(N)
low, high = analysis_filterbank(x, H0, H1)
reconstructed = synthesis_filterbank(low, high, H0, H1)

plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.plot(t, x)
plt.title("Original Signal")
plt.subplot(3, 1, 2)
plt.plot(low, label="Lowpass")
plt.plot(high, label="Highpass")
plt.title("Subbands")
plt.legend()
plt.subplot(3, 1, 3)
plt.plot(reconstructed)
plt.title("Reconstructed Signal")
plt.tight_layout()
plt.show()
```

OUTPUT:


Original Signal
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0
Subbands
Lowpass
Highpass
2
0
−2
0
100
200
300
400
500
Reconstructed Signal
1
0
−1
0
200
400
600
800
1000

Experiment 10

To generate random variables having the following probability distributions (a) Bernoulli (b) Binomial(c) Geometric(d) Poisson (e)Uniform, (f) Gaussian(g) Exponential (h) Laplacian

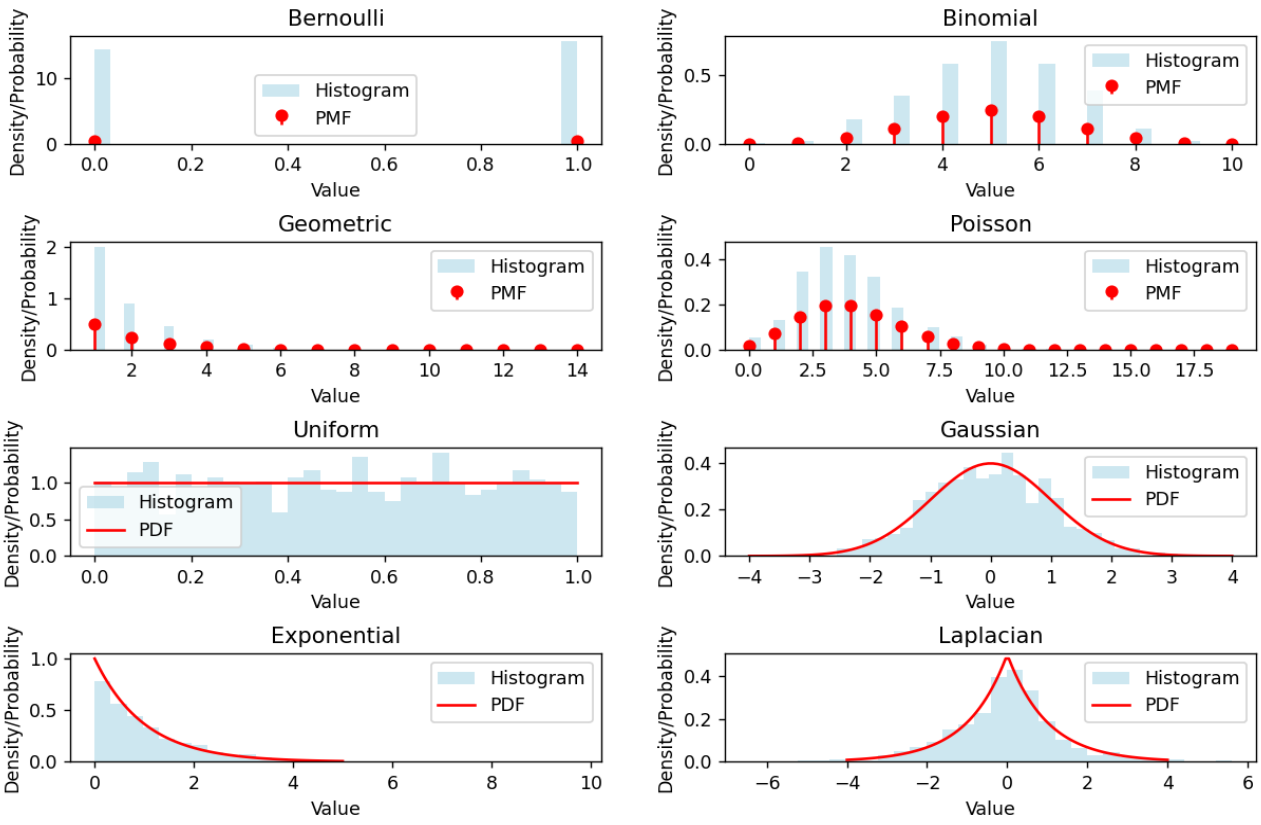
Code:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import bernoulli, binom, geom, poisson, uniform, norm, expon, laplace


# Parameters for the distributions

n = 1000  # Number of samples

p_bernoulli = 0.5

n_trials_binomial, p_binomial = 10, 0.5

p_geometric = 0.5

lambda_poisson = 4

a_uniform, b_uniform = 0, 1

mu_gaussian, sigma_gaussian = 0, 1

lambda_exponential = 1

mu_laplacian, b_laplacian = 0, 1


# Generate random samples

samples = {

    "Bernoulli": np.random.choice([0, 1], size=n, p=[1-p_bernoulli, p_bernoulli]),

    "Binomial": np.random.binomial(n_trials_binomial, p_binomial, size=n),

    "Geometric": np.random.geometric(p_geometric, size=n),

    "Poisson": np.random.poisson(lambda_poisson, size=n),

    "Uniform": np.random.uniform(a_uniform, b_uniform, size=n),

    "Gaussian": np.random.normal(mu_gaussian, sigma_gaussian, size=n),

    "Exponential": np.random.exponential(1/lambda_exponential, size=n),
```

```
    "Laplacian": np.random.laplace(mu_laplacian, b_laplacian, size=n),
}

# x-values for PDFs/PMFs
x_values = {
    "Bernoulli": np.array([0, 1]),
    "Binomial": np.arange(0, n_trials_binomial + 1),
    "Geometric": np.arange(1, 15),
    "Poisson": np.arange(0, 20),
    "Uniform": np.linspace(a_uniform, b_uniform, 100),
    "Gaussian": np.linspace(mu_gaussian - 4 * sigma_gaussian, mu_gaussian + 4 * sigma_gaussian, 100),
    "Exponential": np.linspace(0, 5 / lambda_exponential, 100),
    "Laplacian": np.linspace(mu_laplacian - 4 * b_laplacian, mu_laplacian + 4 * b_laplacian, 100),
}

# Compute theoretical PDFs/PMFs
theoretical = {
    "Bernoulli": bernoulli.pmf(x_values["Bernoulli"], p_bernoulli),
    "Binomial": binom.pmf(x_values["Binomial"], n_trials_binomial, p_binomial),
    "Geometric": geom.pmf(x_values["Geometric"], p_geometric),
    "Poisson": poisson.pmf(x_values["Poisson"], lambda_poisson),
    "Uniform": uniform.pdf(x_values["Uniform"], a_uniform, b_uniform - a_uniform),
    "Gaussian": norm.pdf(x_values["Gaussian"], mu_gaussian, sigma_gaussian),
    "Exponential": expon.pdf(x_values["Exponential"], scale=1 / lambda_exponential),
    "Laplacian": laplace.pdf(x_values["Laplacian"], mu_laplacian, b_laplacian),
}

# Plot histograms with overlays
plt.figure(figsize=(15, 12))
for i, (name, data) in enumerate(samples.items(), 1):
    plt.subplot(4, 2, i)
    plt.hist(data, bins=30, density=True, alpha=0.6, color='lightblue', label="Histogram")
```

```
    x = x_values[name]
    pdf_pmf = theoretical[name]
    if name in ["Bernoulli", "Binomial", "Geometric", "Poisson"]:  # Discrete distributions
        plt.stem(x, pdf_pmf, basefmt=" ", linefmt="r-", markerfmt="ro", label="PMF")
    else:  # Continuous distributions
        plt.plot(x, pdf_pmf, 'r-', label="PDF")
    plt.title(name)
    plt.legend()
    plt.xlabel("Value")
    plt.ylabel("Density/Probability")
plt.tight_layout()
plt.show()
```

OUTPUT:



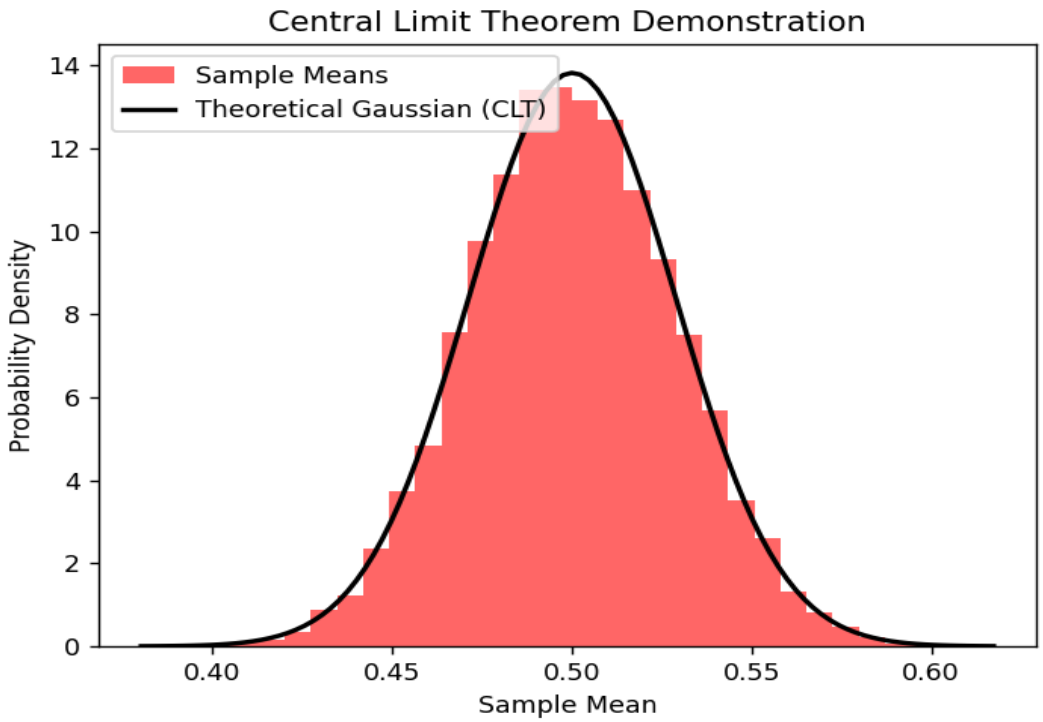
Experiment 11

Central Limit Theorem

Code:

```
import random

import matplotlib.pyplot as plt

import math

# Parameters for the original uniform distribution

lower_bound, upper_bound = 0, 1  # Bounds of the uniform distribution

sample_size = 100              # Number of values in each sample

num_samples = 10000            # Total number of samples

# Generate samples and calculate sample means

samples = [[random.uniform(lower_bound, upper_bound) for _ in range(sample_size)] for _ in
range(num_samples)]

sample_means = [sum(sample) / sample_size for sample in samples]


# Plot the histogram of sample means

plt.hist(sample_means, bins=30, density=True, alpha=0.6, color='g', label='Sample Means')


# Calculate mean and standard deviation of sample means

mu = sum(sample_means) / len(sample_means)

sigma = math.sqrt(sum((x - mu) ** 2 for x in sample_means) / len(sample_means))


# Plot the theoretical Gaussian curve (Central Limit Theorem)

xmin, xmax = plt.xlim()

x = [xmin + i * (xmax - xmin) / 100 for i in range(100)]

p = [1 / (sigma * math.sqrt(2 * math.pi)) * math.exp(-(i - mu) ** 2 / (2 * sigma ** 2)) for i in x]
```

```
plt.plot(x, p, 'k', linewidth=2, label='Theoretical Gaussian (CLT)')
# Display the plot
plt.title('Central Limit Theorem Demonstration')
plt.xlabel('Sample Mean')
plt.ylabel('Probability Density')
plt.legend()
plt.show()
```



Experiment 12

K Nearest Neighbors Algorithm with decision region plots

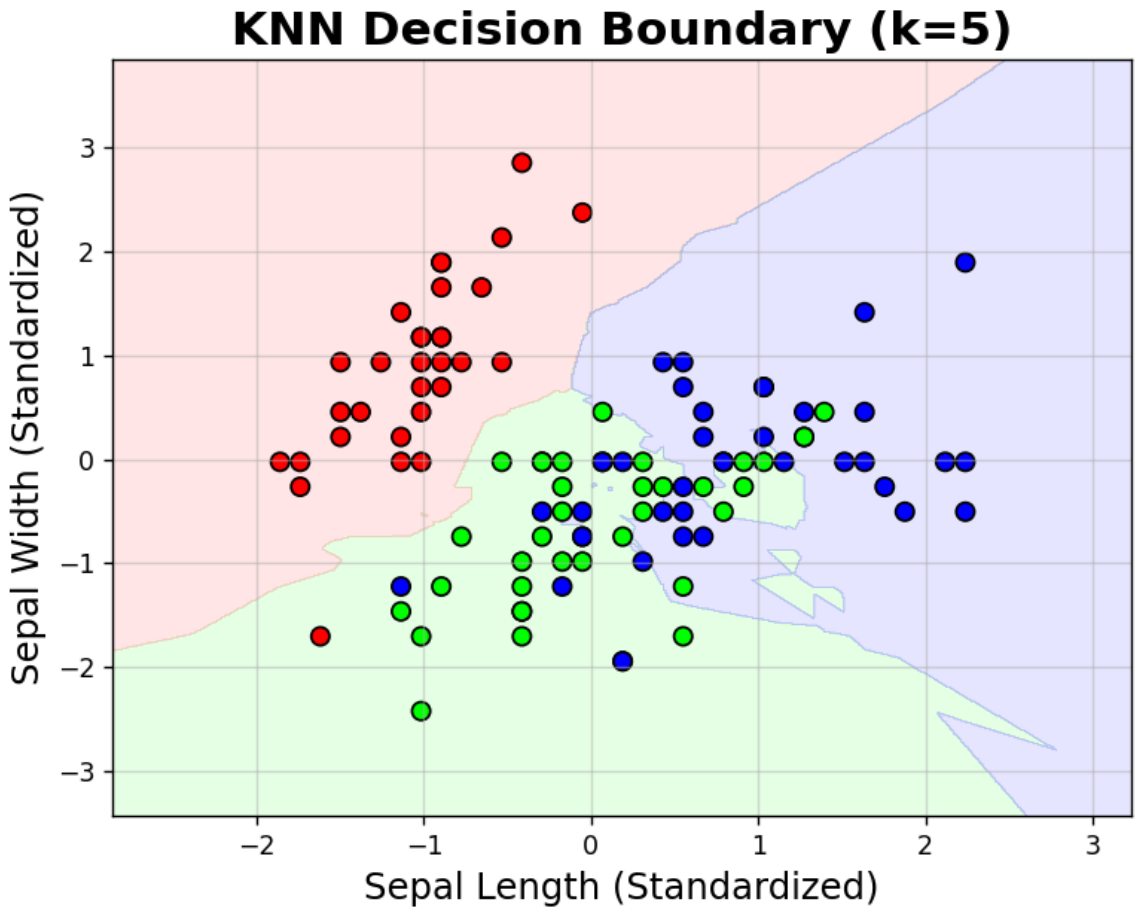
Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from matplotlib.colors import ListedColormap

# Load Iris dataset and select two features for visualization
iris = datasets.load_iris()
X = iris.data[:, :2]  # Sepal length and sepal width
y = iris.target

# Train-test split (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize features (important for KNN)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN classifier
k = 5
```

```
knn = KNeighborsClassifier(n_neighbors=k, weights='uniform')
knn.fit(X_train, y_train)

# Function to plot decision boundary
def plot_decision_boundary(X, y, model, title):
    # Generate a mesh grid for decision regions
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                         np.arange(y_min, y_max, 0.01))

    # Predict class for each point in the grid
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # Create color maps for plotting
    cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
    cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])

    # Plot the decision boundary
    plt.figure(figsize=(10, 8))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=cmap_light)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold, edgecolor='k', s=50)
    plt.title(title, fontsize=18, fontweight='bold')
    plt.xlabel('Sepal Length (Standardized)', fontsize=14)
    plt.ylabel('Sepal Width (Standardized)', fontsize=14)
    plt.grid(alpha=0.5)
    plt.show()

# Plot the decision regions for the training data
plot_decision_boundary(X_train, y_train, knn, title=f"KNN Decision Boundary (k={k})")

# Evaluate accuracy on the test set
```

```
y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f'Accuracy: {accuracy * 100:.2f}%')
```

OUTPUT:



Experiment 13

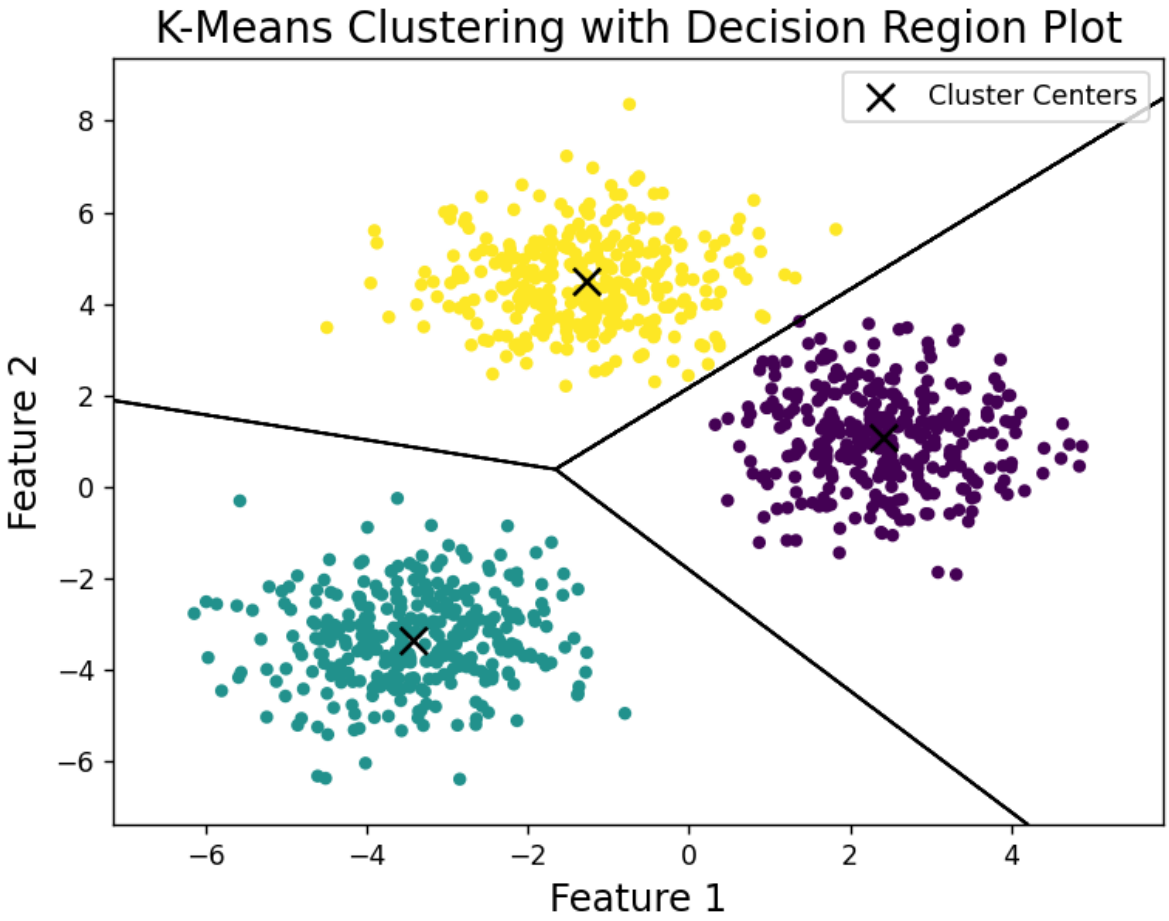
K Means Algorithm with decision region plots

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Parameters
N = 1000  # Number of data points
K = 3     # Number of clusters

# Generate random cluster centers
np.random.seed(42)  # Set seed for reproducibility
C = np.random.rand(K, 2) * 10 - 5  # Cluster centers in range [-5, 5]

# Generate data points around cluster centers
X = np.vstack([C[k] + np.random.randn(N // K, 2) for k in range(K)])

# Apply K-Means clustering
kmeans = KMeans(n_clusters=K, n_init=20, max_iter=300, random_state=42)
kmeans.fit(X)
idx = kmeans.labels_
C = kmeans.cluster_centers_

# Plot data points and cluster centers
plt.figure(figsize=(10, 8))
```

```
plt.scatter(X[:, 0], X[:, 1], c=idx, cmap='viridis', s=15)
plt.scatter(C[:, 0], C[:, 1], c='black', marker='x', s=100, label='Cluster Centers')

# Generate a grid of points to plot decision regions
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
x1, x2 = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))
X_grid = np.c_[x1.ravel(), x2.ravel()]

# Predict cluster index for each point in the grid
idx_grid = kmeans.predict(X_grid)
decision_boundary = idx_grid.reshape(x1.shape)

# Plot decision boundaries
plt.contour(x1, x2, decision_boundary, colors='black', linewidths=1)

# Add title, labels, legend, and display plot
plt.title('K-Means Clustering with Decision Region Plot', fontsize=16)
plt.xlabel('Feature 1', fontsize=14)
plt.ylabel('Feature 2', fontsize=14)
plt.legend()
plt.show()
```

OUTPUT:


K-Means Clustering with Decision Region Plot
Cluster Centers
Feature 2
Feature 1
8
6
4
2
0
−2
−4
−6
−6
−4
−2
0
2
4

Experiment 14

PERCEPTRON LEARNING

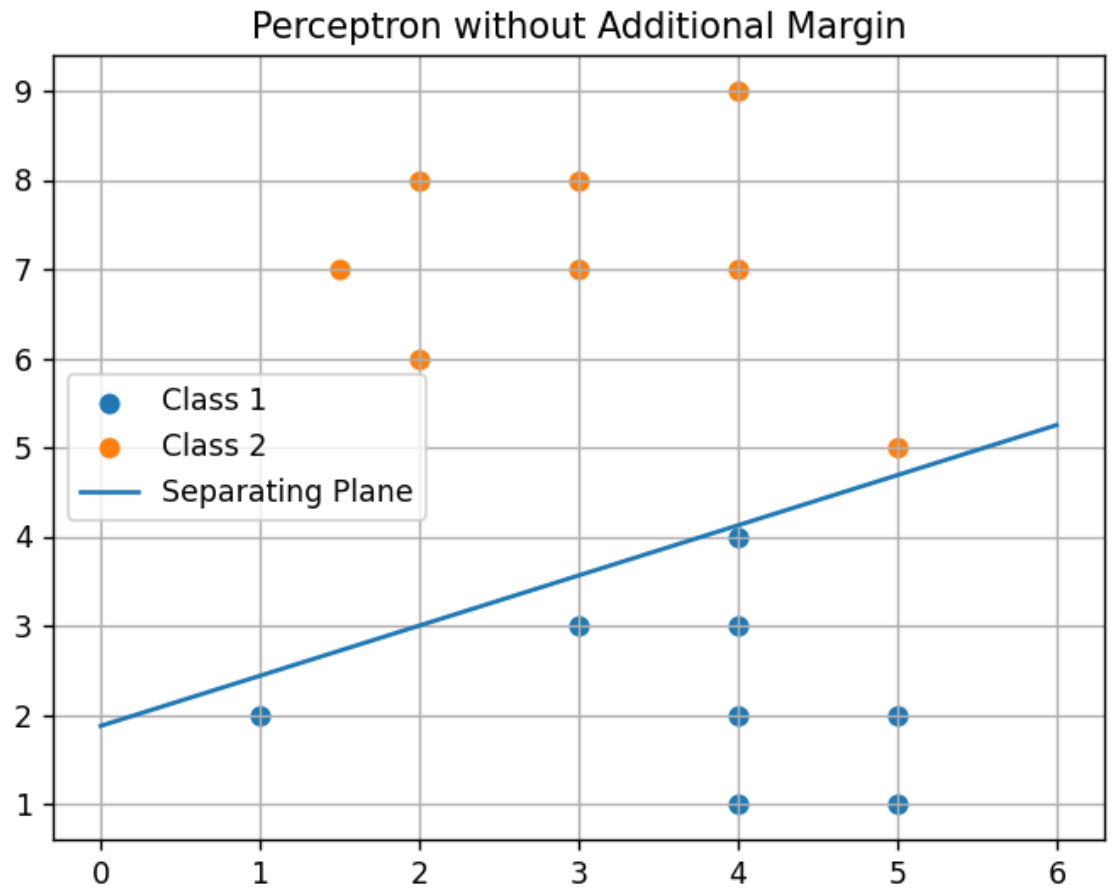
Code:

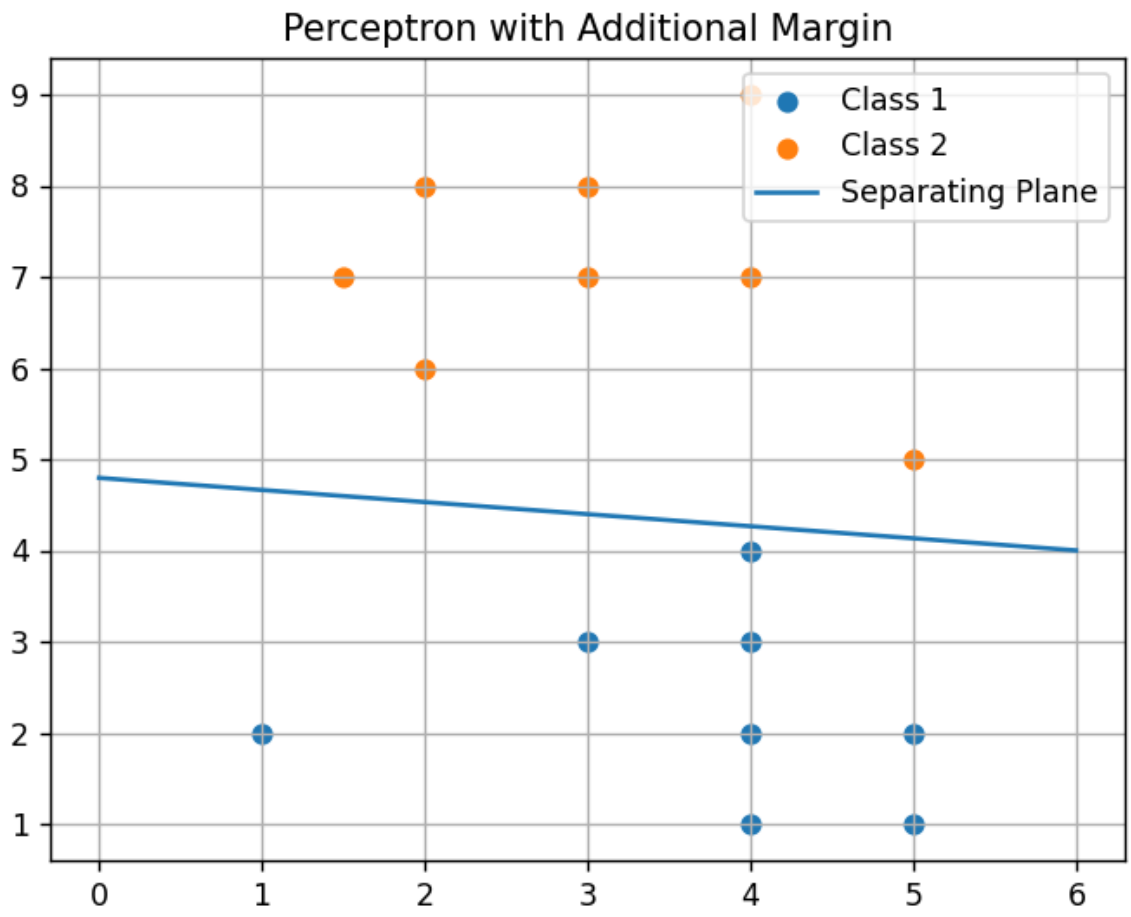
```
import numpy as np
import matplotlib.pyplot as plt


def perceptron_without_margin(class1, class2, eta=1, max_iter=500):
    norm_class2 = -class2
    Y = np.vstack((class1, norm_class2))
    a = np.array([0, 0, 0])
    k = 0

    while k < max_iter:
        prev_a = a.copy()
        for i in range(len(Y)):
            temp = Y[i, :]
            ax = np.dot(a, temp)
            if ax <= 0:
                a = a + eta * temp
        if np.array_equal(prev_a, a):
            break
        k += 1

    return a


def perceptron_with_margin(class1, class2, eta=1, b=2.6, max_iter=500):
    norm_class2 = -class2
    Y = np.vstack((class1, norm_class2))
```

```
    a = np.array([0, 0, 0])
    k = 0

    while k < max_iter:
        prev_a = a.copy()
        for i in range(len(Y)):
            temp = Y[i, :]
            ax = np.dot(a, temp)
            if ax <= b:
                dist_point = b - ax
                a = a + (eta * (dist_point * temp) / np.linalg.norm(temp) ** 2)
        if np.array_equal(prev_a, a):
            break
        k += 1

    return a

def plot_decision_boundary(a, class1, class2, title):
    x1 = np.arange(0, 6.1, 0.1)
    x2 = (-a[0] / a[1]) * x1 - a[2] / a[1]

    plt.scatter(class1[:, 0], class1[:, 1], label='Class 1')
    plt.scatter(class2[:, 0], class2[:, 1], label='Class 2')
    plt.plot(x1, x2, label='Separating Plane')
    plt.grid(True)
    plt.title(title)
    plt.legend()
    plt.show()

# Data for Class 1 and Class 2
class1 = np.array([[1, 2, 1], [3, 3, 1], [4, 1, 1], [4, 2, 1], [4, 3, 1], [4, 4, 1], [5, 1, 1], [5, 2, 1]])
class2 = np.array([[1.5, 7, 1], [2, 6, 1], [2, 8, 1], [3, 7, 1], [3, 8, 1], [4, 7, 1], [4, 9, 1], [5, 5, 1]])
```

```
# Perceptron without additional margin
a_no_margin = perceptron_without_margin(class1, class2)
plot_decision_boundary(a_no_margin, class1, class2, 'Perceptron without Additional Margin')

# Perceptron with additional margin
a_with_margin = perceptron_with_margin(class1, class2)
plot_decision_boundary(a_with_margin, class1, class2, 'Perceptron with Additional Margin')
```

OUTPUT:




Perceptron with Additional Margin
Class 1
Class 2
Separating Plane
9
8
7
6
5
4
3
2
1
0
1
2
3
4
5
6

Experiment 15

Implementation of SVM algorithm for classification applications

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay

# Generate synthetic data with more overlap
X, y = make_blobs(n_samples=500, centers=2, random_state=45, cluster_std=3.0)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train an SVM classifier with a linear kernel
svm = SVC(kernel='linear', C=1.0)
svm.fit(X_train, y_train)

# Make predictions
y_pred = svm.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
# Display confusion matrix
ConfusionMatrixDisplay.from_estimator(svm, X_test, y_test)
```

```
plt.show()


# Visualize decision boundary
def plot_decision_boundary(model, X, y):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max, 0.01))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)


    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)
    plt.title("SVM Decision Boundary")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()
plot_decision_boundary(svm, X, y)
```

OUTPUT: